Ejercicio 1:

1.1 Si se aplica el algoritmo A^* con una función heurística h_1^*, tal que $h_1^*(n) = 0$ para todos los nodos n, entonces

(a) la función h_1^* es optimista.
(b) todas las funciones heurísticas optimistas h_2^* son más informadas que h_1^*.
(c) el algoritmo A^* expande los mismos nodos que la búsqueda de coste uniforme.
(d) no existe ninguna función heurística optimista h_2^* con la que el algoritmo A^* expande menos nodos que con h_1^*.

1.2 Suponga que para un problema a resolver con el algoritmo A^* se dispone de varias funciones heurísticas optimistas h_1^*, ..., h_l^*, todas ellas de fácil evaluación. ¿Cuáles de las siguientes afirmaciones son correctas y cuáles son falsas?

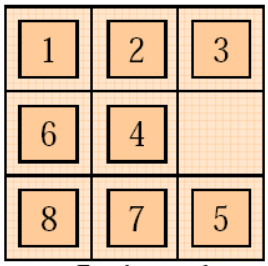
(a) Si hay una función h_k^* que es más informada que h_i^*, entonces se puede prescindir de h_i^* en el proceso de búsqueda.
(b) Si para un nodo n se cumple que $h_i^*(n) < h_k^*(n)$, entonces se puede prescindir de h_i^* en todo el proceso de búsqueda.
(c) Para cada nodo n es conveniente elegir la función h_i^* de máximo valor.
(d) Si h_i^* *no* es consistente, entonces se puede prescindir de h_i^* en el proceso de búsqueda.

1.3 ¿Cuáles de las siguientes afirmaciones acerca de la búsqueda en línea son ciertas y cuáles son falsas?

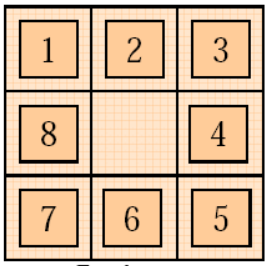
(a) Se intercala una fase de búsqueda (elección de acciones) con una fase de acción/percepción.
(b) La búsqueda en línea siempre es óptima y completa.
(c) Es conveniente aplicar la búsqueda con horizonte cuando el espacio de búsqueda es demasiado grande para realizar una *única* búsqueda A^*.
(d) Una búsqueda en línea es más eficiente cuanto mayor sea su índice competitivo (coste del camino real entre coste del camino óptimo).

Ejercicio 2:

Considere el 8-puzzle cuyo estado inicial y estado meta se muestran en la siguiente figura:



Estado inicial



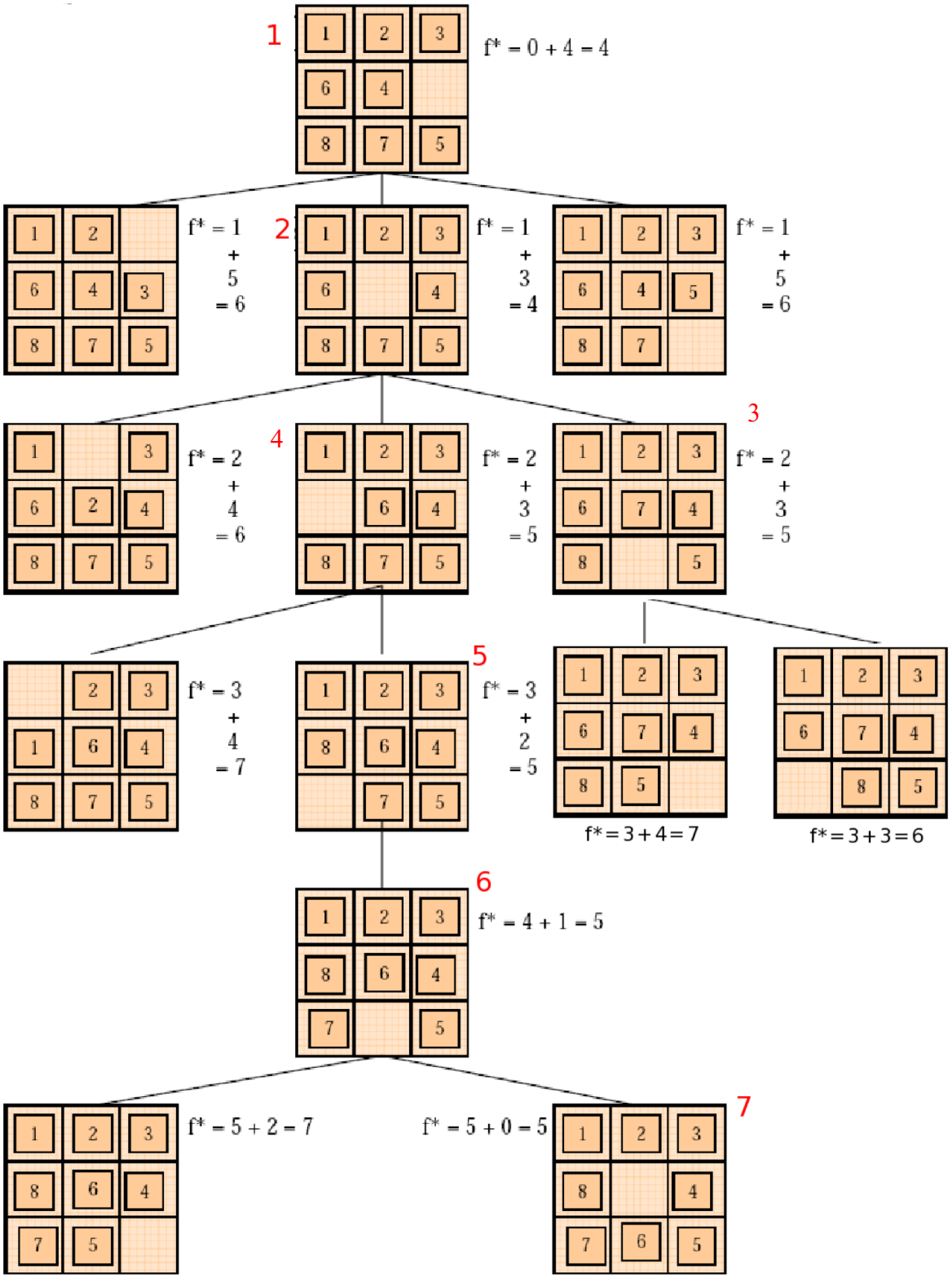
Estado meta

Desarrolle el árbol de búsqueda que expande el algoritmo A^* utilizando las siguientes heurísticas. Evite ciclos generales, indique el orden de expansión de los estados y muestre en cada paso los valores de h^*, g, y f^*. Cuando el algoritmo puede elegir entre varios nodos para ser expandidos, elige aquella acción que NO está en el camino óptimo (es decir, asuma que el algoritmo elige el "peor caso").

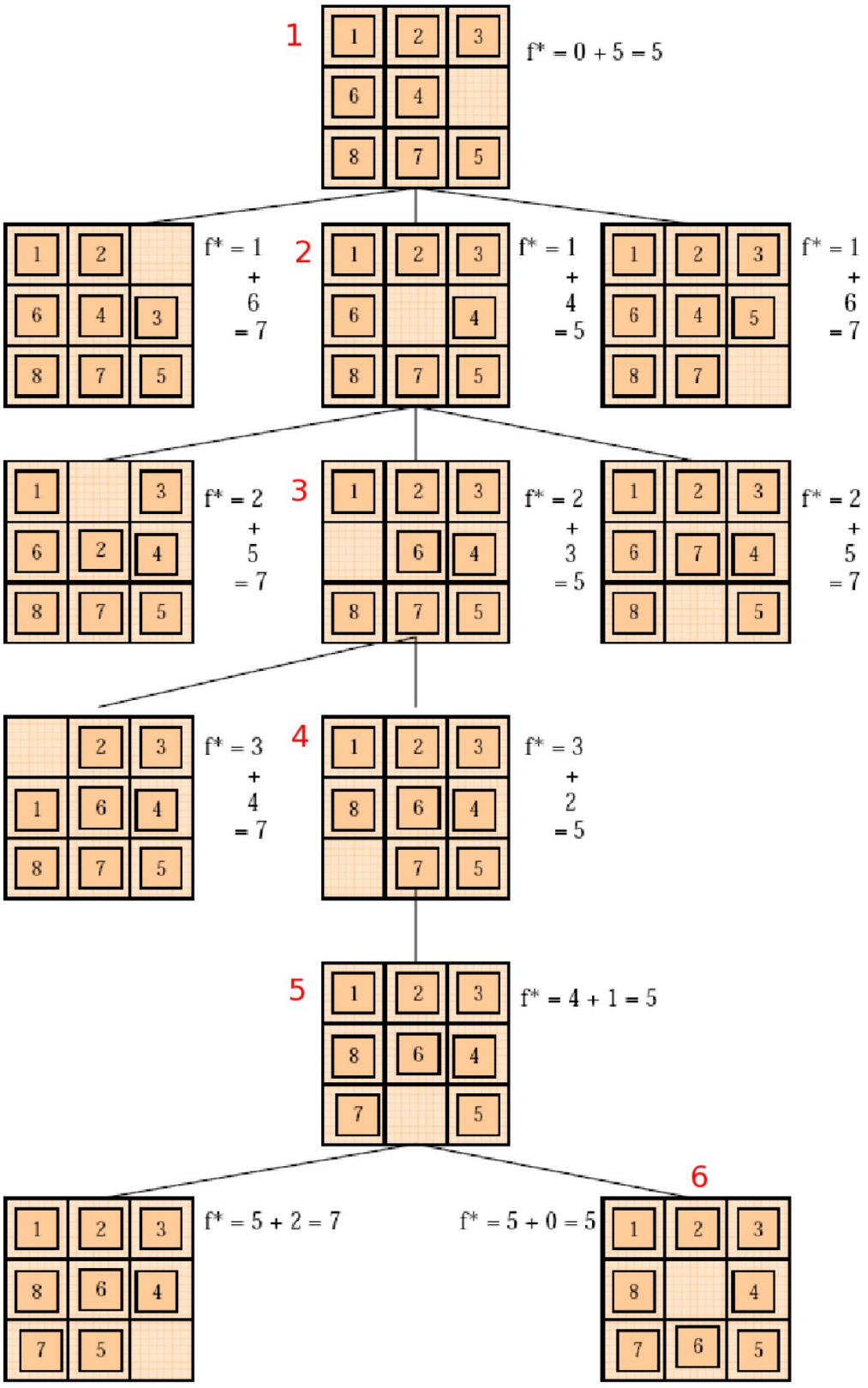
a) $h_a^*(n)$ = número de piezas descolocados en n respecto al estado meta

b) $h_c^*(n)$ = suma de las distancias de Manhattan de las piezas en n respecto al estado meta

Solución a:

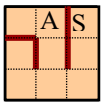

1
f* = 0 + 4 = 4
2
f* = 1 + 5 = 6
f* = 1 + 3 = 4
f* = 1 + 5 = 6
3
4
f* = 2 + 4 = 6
f* = 2 + 3 = 5
f* = 2 + 3 = 5
5
f* = 3 + 4 = 7
f* = 3 + 2 = 5
f*=3+4=7
f*=3+3=6
6
f* = 4 + 1 = 5
7
f* = 5 + 2 = 7
f* = 5 + 0 = 5

Solución b:


1
f* = 0 + 5 = 5
f* = 1 + 6 = 7
2
f* = 1 + 4 = 5
f* = 1 + 6 = 7
f* = 2 + 5 = 7
3
f* = 2 + 3 = 5
f* = 2 + 5 = 7
f* = 3 + 4 = 7
4
f* = 3 + 2 = 5
5
f* = 4 + 1 = 5
f* = 5 + 2 = 7
6
f* = 5 + 0 = 5

Ejercicio 3:

Considere el problema el laberinto que se presenta en la siguiente figura.

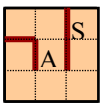


El agente A tiene el objetivo de encontrar la salida S. Las únicas acciones de las que el agente dispone son los movimientos (derecha, arriba, abajo, e izquierda) del cuadrado en el que se encuentra el agente en un momento dado a un cuadrado adyacente. Sin embargo, cada una de estas acciones sólo es posible si en la dirección correspondiente no existe una barrera ni se saldría del tablero. Cada acción tiene un **coste de una unidad**. De antemano, el agente **conoce el mapa** del laberinto **y la posición de la salida**, pero **no las posiciones de las barreras**. Además el agente es capaz de identificar en cada momento su propia posición en el mapa.

a) Define una función heurística h^* para este problema. ¿Es su función h^* optimista y/o consistente? ¡Justifique brevemente su propuesta!

b) Suponiendo la posición inicial del agente que se indica en la figura arriba, aplique la *búsqueda A** con su función heurística h^* a este problema. Desarrolle el árbol de búsqueda suponiendo que **no se evitan estados repetidos**. Indique el orden en el que se expanden los nodos, así como los valores de g, h^* y f^* para cada nodo del árbol de búsqueda. **(Si al expandir un nodo hay que elegir aleatoriamente entre varios, expanda preferiblemente primero un nodo que está más cerca de un nodo meta.)**

c) Suponga el siguiente estado inicial:



Suponga que el agente **no tiene ninguna información heurística** inicial respecto a las distancias de las distintas posiciones en el tablero hacia la salida. Aplique la *búsqueda A** con el aprendizaje de la función heurística h^* a este problema. Desarrolle el árbol de búsqueda suponiendo que **no se evitan estados repetidos**. Indique el orden en el que se expanden los nodos, así como los valores de g, h^* y f^* para cada nodo del árbol de búsqueda. Además, anote los valores de la función heurística h* de cada nodo en una tabla **de tal modo que se aprecian los cambios de estos valores** a lo largo de la ejecución del algoritmo. **(Si al expandir un nodo hay que elegir aleatoriamente entre varios, expanda preferiblemente primero el nodo que está más cerca de un nodo meta.)**

Solución 3a)

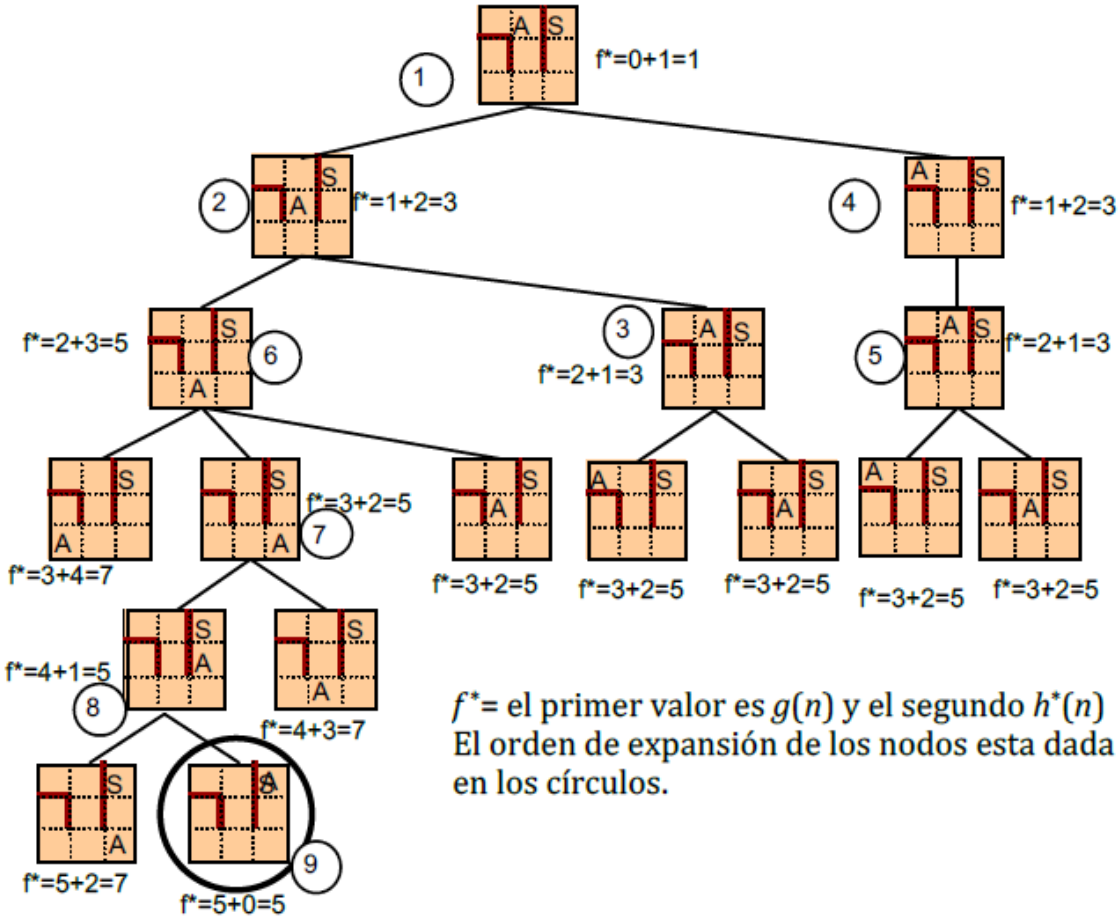
Una función heurística evidente es la que mide la distancia mínima del agente hasta la salida (en número mínimo de cuadrados que tendría que recorrer) sin tener en cuenta las barreras. Para el estado de la figura del enunciado el valor de h^* sería 1.

Esta función es optimista, ya que en el caso real (incluidas las barreras) el número de cuadrados a recorrer hasta la salida sería siempre igual o mayor que el valor de h^*.

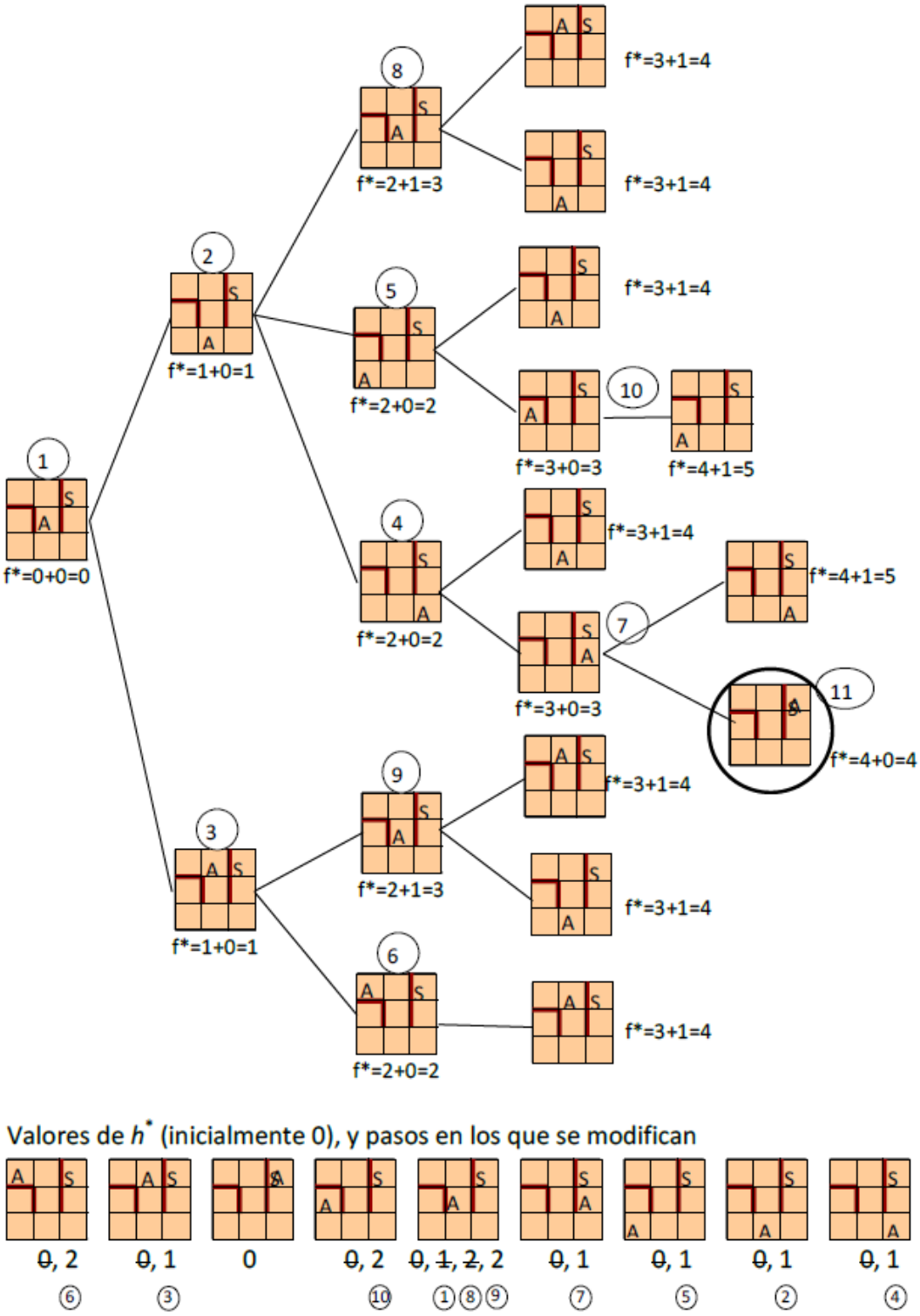
Respecto a la consistencia, una función h^* es consistente sii para todo nodo n_i y todo hijo n_j de n_i se cumple que $h^*(n_i) - h^*(n_j) \leq c(n_i,n_j)$. Esta condición se cumple para la función elegida como se ve a continuación:

- para todo nodo n_i y todo hijo n_j de n_i se cumple que $c(n_i,n_j)=1$, ya que cada operación vale una unidad
- para todo nodo n_i y todo hijo n_j de n_i se cumple que $h^*(n_i) - h^*(n_j) \in \{1,-1\}$ (eso es así porque para cualquiera dos estados adyacentes, uno es siempre una unidad más "cerca" de la salida que el otro)
- por tanto para todo nodo n_i y todo hijo n_j de n_i se cumple necesariamente la condición $h^*(n_i) - h^*(n_j) \leq c(n_i,n_j)$, por lo que h^* es consistente.

Solución 3b)



Solución 3c)



Nótese que, en caso de aplicar actualización de valores f^* en las hojas, no se expandiría el nodo 9 puesto que, al aprender un nuevo valor h^* después de expandir 8, su valor f^* cambiaría a 2+2=4.

Ejercicio 4:

En juego de los "cuadrados latinos" se parte de un tablero 3 × 3 vacío. En cada posición vamos colocamos números del 1 al 9, ninguno de los cuales puede repetirse. El objetivo es tener el tablero completo, es decir, un número en cada posición del mismo, y es necesario que el valor de la suma de las filas, columnas y diagonales sea siempre el mismo valor: 15. Un ejemplo en el cual tenemos el tablero completo, se han utilizado todos los números pero no se consigue el objetivo indicado está en la siguiente figura. En este ejemplo sólo una diagonal y la última fila cumplen que la suma de sus números es 15.

1	9	2
7	5	6
8	4	3

a) Defina una representación eficiente para el juego de los cuadrados latinos 3×3, especificando el conjunto de estados, el estado inicial, y las operaciones permitidas en cada estado.

b) Considere el estado inicial que se muestra a continuación donde tenemos seis posiciones ocupadas, tres libres, y sólo una diagonal cumple que la suma de sus números es 15. Cada paso tiene coste uno.

2		4
	5	3
6	1	

Asimismo, considere la siguiente función heurística definida para cualquier estado n del tablero:

$h^*(n)$ = el número de filas + el número de columnas + el número de diagonales en el estado n que no cumplen que la suma de sus números es 15.

Por ejemplo, el valor de h^* de la configuración de la primera figura es 6, mientras que el valor de h^* para la configuración de la segunda figura es 7.

Desarrolle el árbol de búsqueda que genera el algoritmo A^* para este problema. Indique el orden en el que se expanden los nodos, los valores de g, h^* y f^* para cada nodo del árbol de búsqueda, y la evolución de la lista *abierta*.

c) ¿La función h^* es optimista y/o consistente? ¿El algoritmo A^* encuentra siempre la solución de menor coste? Razone brevemente sus respuestas.

Solución:

a) Sea $N = \{1,2,3,4,5,6,7,8,9\}$ y $T = \{_\} \cup N$ un conjunto de símbolos. El conjunto de estados S viene dado por todos los vectores de longitud 9 sobre T en los que no se repite ningún número:

$$S = \{v \in T^9 |\ \ \forall i,j \in N,\ i \neq j.\ \ v_i \neq v_j \vee v_i = _\}$$

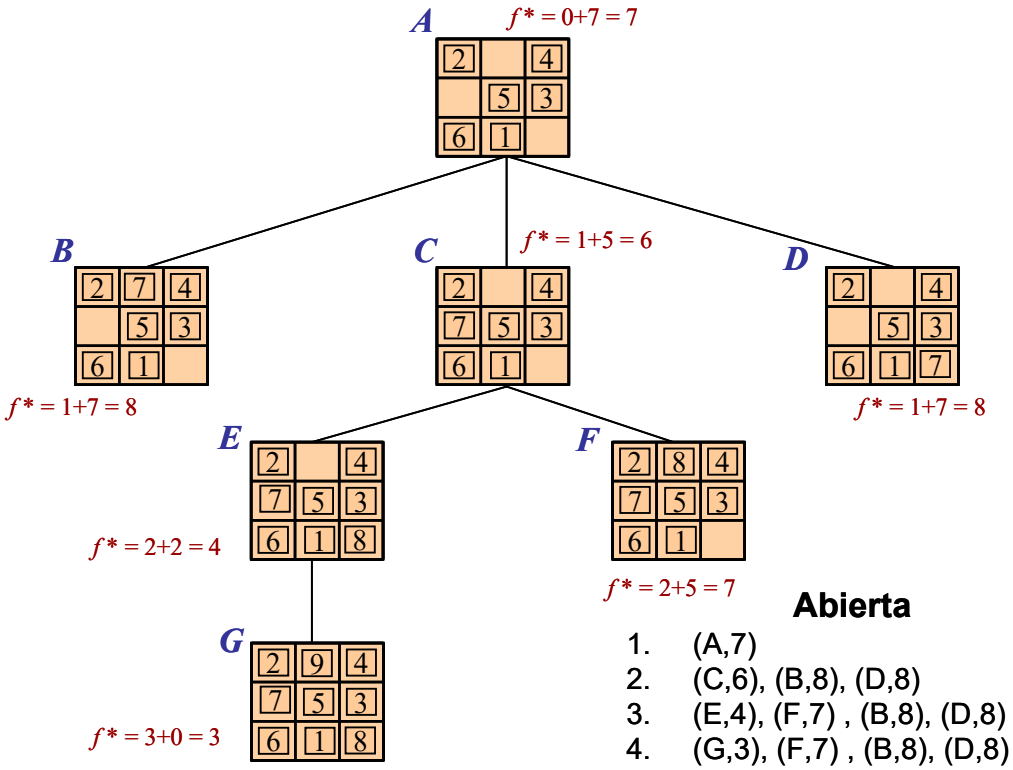
El estado inicial es s_0=(_,_,_,_,_,_,_,_,_).

El orden en el que se añaden números al tablero es irrelevante, por lo que en cada estado decidimos utilizar sólo el menor de los números no usados hasta el momento. Sea n' el número mínimo no utilizado en un estado s $\quad n'_s = \min_{n \in N}\{\forall i \in N.\ n \neq s_i\}$

El conjunto de sucesores de un estado s se genera añadiendo n' en cada "hueco" del tablero

$$expandir(s) = \{s' \in S |\ \exists s_1, s_2 \in T^*.\ \ (s = s_1_s_2) \wedge (s' = s_1 n'_s s_2)\}$$

b)



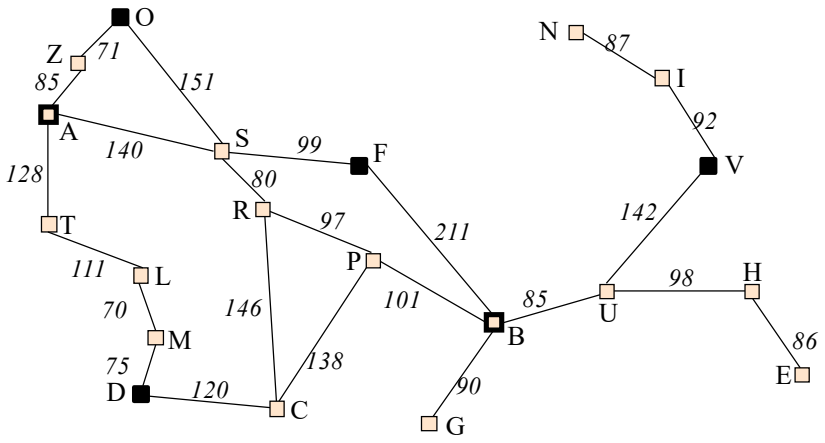
Abierta

1. (A,7)
2. (C,6), (B,8), (D,8)
3. (E,4), (F,7) , (B,8), (D,8)
4. (G,3), (F,7) , (B,8), (D,8)

c) La función heurística no es optimista, como se ve en el apartado b a través del nodo E: $h^*(E)$=2 > h (E)=1. Puesto que no es optimista, tampoco es consistente. Sin embargo, todas las soluciones tienen el mismo coste (si se parte del tablero vacío, el coste de cualquier solución es 9) por lo que, trivialmente, el algoritmo A* es óptimo para el juego de los cuadrados latinos.

Ejercicio 5:

El grafo que se muestra a continuación representa un esquema de un mapa de carreteras. Los nodos están etiquetados con el nombre de ciudades, y los arcos con la distancia por carretera entre dichas ciudades. La función h^* que se muestra al lado indica la distancia aérea entre cualquier ciudad y la ciudad de *B*. Suponga que nuestro agente dispone de un coche eléctrico con una novedosa batería, que le permite recorrer 320km sin recargar. Sólo puede recargar en ciudades con estación de recarga pública, los cuales se indican en el mapa por los cuadrados rellenos (ciudades *D*, *F*, *O*, y *V*). Inicialmente el agente se encuentra en la ciudad *A* con la batería completamente cargada, y desea trasladarse a *B* por la ruta más corta posible (e.d. sin quedarse tirado por el camino con la batería vacía). Conoce la red de carreteras, la posición de las estaciones de recarga, y las características de su vehículo, por lo que en cada momento sabe a qué ciudades vecinas puede ir dado el estado de carga de su batería. El agente decide aplicar la búsqueda A^* para resolver el problema.



	h^*
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

a) Represente el problema como espacio de estados, definiendo el conjunto de estados *S*, el estado inicial s_0, los estados meta (a través de la función *meta?*), el coste $c(s_i,s_j)$ para ir de una ciudad s_i a la ciudad vecina s_j, así como la función *expandir* (para dichas definiciones, puede suponer que existe un predicado binario *adyacente* que indica que dos ciudades son vecinas, un predicado unario *carga* que indica si hay una estación de recarga en una ciudad, y una función binaria *d* que proporciona la distancia entre dos ciudades vecinas de acuerdo con el mapa).

b) Desarrolle el árbol de búsqueda que genera el algoritmo A^* (no se filtran ciclos de ningún tipo). Indique el orden en el que se expanden los nodos, los valores de *g*, h^*, y f^* de cada nodo del árbol de búsqueda, así como el estado de la lista *abierta* en cada paso del algoritmo.

c) ¿Para la representación del apartado (a), la función heurística h^* (distancia área entre ciudades) sería optimista? Justifique brevemente su opinión.

Solución:

a) Sea P el conjunto de ciudades

- Conjunto de estados: $S = P \times \aleph$
- Estado inicial: $s_0 = (A,320)$
- Determinación si un estado es una meta:

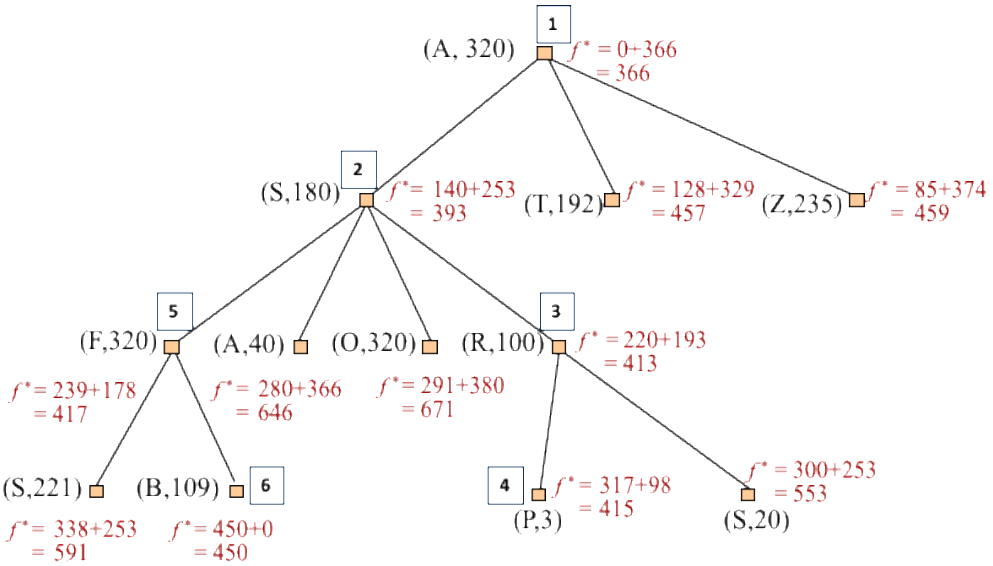
$$meta(p,n) = \begin{cases} true & si\ p = B \\ false & de\ lo\ contrario \end{cases}$$

- Determinación de los sucesores inmediatos de un estado:

$$expandir((p,n)) = \{(p', n - d(p,p') | adyacente(p,p') \wedge n \geq d(p,p') \wedge \neg carga(p')\} \cup \{(p', 320) | adyacente(p,p') \wedge n \geq d(p,p') \wedge carga(p')\}$$

- Coste de transitar de un estado a otro:
  $c((p_1,n_1),(p_2,n_2)) = d(p_1,p_2)$ si $(p_2,n_2) \in expandir((p_1,n_1))$

b)



Lista abierta:

1. [(A,320),366]
2. [(S,180),393] [(T,192),457] [(Z,235),459]
3. [(R,100),413] [(F,320),417] [(T,192),457] [(Z,235),459] [(A,40),646] [(O,320),671]
4. [(P,3),415)] [(F,320),417] [(T,192),457] [(Z,235),459] [(S,20),553] [(A,40),646] [(O,320),671]
5. [(F,320),417] [(T,192),457] [(Z,235),459] [(S,20),553] [(A,40),646] [(O,320),671]
6. [(B,109),450] [(T,192),457] [(Z,235),459] [(S,20),553] [(S,221),591] [(A,40),646] [(O,320),671]

c) Como se comprueba fácilmente, la función heurística h^* es optimista para el problema habitual de encontrar el camino más corto en una red de carreteras (representa la distancia aérea). Dicho problema se puede considerar una versión relajada del problema del ejercicio: todas las rutas que se pueden generar en el problema del ejercicio se pueden generar también en el problema habitual de encontrar el camino más corto (y algunas más). Por tanto, una función heurística que es optimista para dicho problema, también lo es para el problema más restrictivo del ejercicio.